



INDIAN INSTITUTE OF TECHNOLOGY MADRAS

Software Engineering — Project Milestone Report

MILESTONE 3

Sprint 1 Report

SahyogRide

An AI-Assisted Community Ride & Seat-Booking Platform

Team 078

Team Code MAY2026-Team-078

Team Lead Vibhav Shukla

Team Members

Name	Email	Roll No
Vibhav Shukla (Team Lead)	23f2003054@ds.study.iitm.ac.in	23f2003054
Siddharth Kumar Yadav	23f1001362@ds.study.iitm.ac.in	23f1001362
Nilesh Mehra	22f3002392@ds.study.iitm.ac.in	22f3002392
Navneet Kumar Yadav	23f2002092@ds.study.iitm.ac.in	23f2002092
Aakash Kumar Katha	21f1004110@ds.study.iitm.ac.in	21f1004110

AI Features in SahyogRide

SahyogRide integrates 3 AI-powered features built on top of the FastAPI backend:

#	Feature	Endpoint	Description
1	Semantic Trip Search	GET /ai/search?q=...	Uses pgvector embeddings to match natural language queries (e.g., "hospital trip tomorrow morning") to available trips. Falls back to keyword search if AI is disabled.
2	AI Trip Triage	Internal service	After a reservation is confirmed, an AI service automatically categorises the trip purpose and flags trips with unusually high demand for coordinator attention.
3	Embedding Backfill	backfill_embeddings.py	Batch script that generates and stores vector embeddings for all existing trips in the PostgreSQL database using the pgvector extension, enabling future semantic search.

Note: All AI features are isolated and optional — the app fully functions (register → search → hold → confirm → cancel) even when `AI_ENABLED=false` in the environment config.

Section 1: API Test Cases

Note: TC-01 to TC-05 & TC-07: Passing tests. TC-06: Failing test that revealed a role enforcement bug.

✓ TC-01 — User Registration (Success)

Field	Details
API Endpoint	POST /auth/register
User Story	SAHYOG-04: Register & Login

Input:

```
{
  "name": "Asha Rao",
  "email": "asha.rider@example.com",
  "password": "password123",
  "role": "rider"
}
```

Expected Output:

```
HTTP Status: 201 Created
{
  "token": "<jwt_string>",
  "user": {
    "id": 1,
    "name": "Asha Rao",
    "email": "asha.rider@example.com",
    "role": "rider"
  }
}
```

Actual Output:

```
HTTP Status: 201 Created
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOi...",
  "user": {
    "id": 1,
    "name": "Asha Rao",
    "email": "asha.rider@example.com",
    "role": "rider"
  }
}
```

Result: PASS

```
tests/test_auth.py
# tests/test_auth.py
import uuid
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def unique_email(prefix: str) -> str:
    return f"{prefix}.{uuid.uuid4().hex[:12]}@example.com"

def test_register_returns_token_and_user():
```

```

"""TC-01: Successful rider registration returns JWT and user."""
res = client.post(
    "/auth/register",
    json={
        "name": "Asha Rao", "email": unique_email("rider"),
        "password": "password123",
        "role": "rider",
    },
)
assert res.status_code == 201
body = res.json()
assert body["user"]["role"] == "rider"
assert body["token"] # JWT must be non-empty

```

✓ TC-02 — Duplicate Email Registration (Conflict)

Field	Details
API Endpoint	POST /auth/register
User Story	SAHYOG-04: Register & Login

Input (same email used again):

```

{
  "name": "Duplicate User",
  "email": "asha.rider@example.com",
  "password": "password123",
  "role": "rider"
}

```

Expected / Actual Output:

```

HTTP Status: 409 Conflict
{
  "error": {
    "code": "EMAIL_TAKEN",
    "message": "An account with this email already exists.",
    "field": "email"
  }
}

```

Result: PASS

```

tests/test_auth.py
# tests/test_auth.py
def test_register_duplicate_email_returns_email_taken():
    """TC-02: Registering with an already-used email returns 409."""
    email = unique_email("dup")
    # First registration
    client.post(
        "/auth/register",
        json={"name": "First", "email": email, "password": "password123", "role": "rider"},
    )
    # Second registration with same email
    res = client.post(
        "/auth/register",
        json={"name": "Duplicate", "email": email, "password": "password123", "role":
"rider"},
    )
    assert res.status_code == 409

```

```
assert res.json()["error"]["code"] == "EMAIL_TAKEN"
```

✓ TC-03 — Coordinator Creates Trip (Success)

Field	Details
API Endpoint	POST /trips
User Story	SAHYOG-05: Coordinator creates trip

Input (Header: Authorization: Bearer <coordinator_jwt>):

```
{
  "origin": "City Hospital",
  "destination": "Railway Station",
  "departure_time": "2026-08-10T09:30:00Z",
  "total_seats": 4,
  "purpose": "medical"
}
```

Expected Output:

```
HTTP Status: 201 Created
{
  "id": 1,
  "origin": "City Hospital",
  "destination": "Railway Station",
  "departure_time": "2026-08-10T09:30:00Z",
  "total_seats": 4,
  "purpose": "medical",
  "coordinator_id": 2
}
```

Actual Output:

```
HTTP Status: 201 Created
{
  "id": 1,
  "origin": "City Hospital",
  "destination": "Railway Station",
  "departure_time": "2026-08-10T09:30:00Z",
  "total_seats": 4,
  "purpose": "medical",
  "coordinator_id": 2,
  "created_at": "2026-08-02T08:45:12Z"
}
```

Result: PASS

```
tests/test_trips.py
# tests/test_trips.py
def test_coordinator_can_create_trip():
    """TC-03: A coordinator can successfully create a trip."""
    coord = register("coordinator")
    token = coord["token"]
    res = client.post(
        "/trips",
        json={
            "origin": "City Hospital",
            "destination": "Railway Station",
            "departure_time": "2026-08-10T09:30:00Z",
```

```

        "total_seats": 4,
        "purpose": "medical",
    },
    headers={"Authorization": f"Bearer {token}"},
)
assert res.status_code == 201
body = res.json()
assert body["origin"] == "City Hospital"
assert body["total_seats"] == 4

```

✓ TC-04 — Hold a Seat (Success)

Field	Details
API Endpoint	POST /holds
User Story	SAHYOG-15, SAHYOG-16: Hold seat (concurrency-safe)

Input (Header: Authorization: Bearer <rider_jwt>):

```
{ "seat_id": 3 }
```

Expected Output:

```

HTTP Status: 201 Created
{
  "id": 7,
  "seat_id": 3,
  "rider_id": 5,
  "expires_at": "2026-08-10T09:35:00Z",
  "status": "held"
}
# Seat status in DB changes from available -> held

```

Actual Output:

```

HTTP Status: 201 Created
{
  "id": 7,
  "seat_id": 3,
  "rider_id": 5,
  "expires_at": "2026-08-10T09:35:00Z",
  "status": "held"
}
# Seat status verified in DB as held

```

Result: PASS

```

tests/test_holds.py
# tests/test_holds.py
from app.database import SessionLocal
from app.models import Seat, SeatStatus

def test_hold_seat_sets_status_to_held():
    """TC-04: Holding an available seat marks it held in the DB."""
    coord = register("coordinator")
    rider = register("rider")
    trip = create_trip(coord["token"], total_seats=4)
    seat_id = seat_ids_for(trip["id"])[0]

    res = client.post(

```

```

        "/holds",
        json={"seat_id": seat_id},
        headers=auth_header(rider["token"]),
    )
    assert res.status_code == 201
    body = res.json()
    assert body["seat_id"] == seat_id
    assert body["status"] == "held"

    # Verify seat status changed in DB
    db = SessionLocal()
    seat = db.query(Seat).filter(Seat.id == seat_id).first()
    assert seat.status == SeatStatus.HELD
    db.close()

```

✓ TC-05 — Confirm Reservation (Success)

Field	Details
API Endpoint	POST /reservations
User Story	SAHYOG-18: Confirm hold into reservation

Input (Header: Authorization: Bearer <rider_jwt>):

```
{ "hold_id": 7 }
```

Expected Output:

```

HTTP Status: 201 Created
{
  "id": 12,
  "seat_id": 3,
  "rider_id": 5,
  "trip_id": 1,
  "status": "confirmed",
  "confirmed_at": "2026-08-10T09:32:00Z"
}
# Seat DB status -> "reserved"

```

Actual Output:

```

HTTP Status: 201 Created
{
  "id": 12,
  "seat_id": 3,
  "rider_id": 5,
  "trip_id": 1,
  "status": "confirmed",
  "confirmed_at": "2026-08-02T09:05:31Z"
}
# Seat DB status -> "reserved" (verified)

```

Result: PASS

```

tests/test_reservations.py
# tests/test_reservations.py
def test_confirm_reservation_marks_seat_reserved():
    """TC-05: Confirming a valid hold creates a confirmed reservation."""
    coord = register("coordinator")
    rider = register("rider")

```

```

trip = create_trip(coord["token"], total_seats=4)
seat_id = seat_ids_for(trip["id"])[0]

hold = client.post(
    "/holds",
    json={"seat_id": seat_id},
    headers=auth_header(rider["token"]),
)
res = client.post(
    "/reservations",
    json={"hold_id": hold.json()["id"]},
    headers=auth_header(rider["token"]),
)
assert res.status_code == 201
assert res.json()["status"] == "confirmed"
assert res.json()["seat_id"] == seat_id

```

✗ TC-06 — Rider Creates Trip (Role Enforcement Bug Found)

Field	Details
API Endpoint	POST /trips
User Story	SAHYOG-05: Only coordinators can create trips
Bug context	During early development, the require_role dependency was not properly wired on the POST /trips endpoint, so riders could incorrectly create trips. This test helped catch that bug.

Input (Header: Authorization: Bearer <rider_jwt> — rider, not coordinator):

```

{
  "origin": "Bus Stand",
  "destination": "College",
  "departure_time": "2026-08-15T08:00:00Z",
  "total_seats": 6,
  "purpose": "exam"
}

```

Expected Output:

```

HTTP Status: 403 Forbidden
{
  "error": {
    "code": "FORBIDDEN_ROLE",
    "message": "You don't have permission to do this.",
    "field": null
  }
}

```

Actual Output (before fix):

```

HTTP Status: 201 Created  # Rider was incorrectly allowed to create a trip!
{
  "id": 3,
  "origin": "Bus Stand",
  "destination": "College",
  "total_seats": 6
}

```


Result: FAIL — Test revealed missing role-guard on POST /trips. Fixed by adding require_role(UserRole.COORDINATOR) dependency.

```
tests/test_trips.py
# tests/test_trips.py
def test_rider_cannot_create_trip():
    """TC-06: FAILING TEST -- Rider attempting to create a trip.

    This test initially FAILED because the require_role dependency was missing
    from the POST /trips endpoint, allowing riders to create trips.
    Fix applied: added coordinator: User = Depends(require_role(...))
    to the create_trip_endpoint function signature in routes/trips.py.
    """
    rider = register("rider")
    res = client.post(
        "/trips",
        json={
            "origin": "Bus Stand",
            "destination": "College",
            "departure_time": "2026-08-15T08:00:00Z",
            "total_seats": 6,
            "purpose": "exam",
        },
        headers=auth_header(rider["token"]),
    )
    # Expected: 403, Actual (pre-fix): 201
    assert res.status_code == 403 # <-- This assertion failed pre-fix
    assert res.json()["error"]["code"] == "FORBIDDEN_ROLE"
```

✓ TC-07 — Block Double Hold on Same Trip (ALREADY_HOLDING)

Field	Details
API Endpoint	POST /holds
User Story	SAHYOG-15: One hold per rider per trip

Step 1 — First hold (succeeds): { "seat_id": 1 } → HTTP 201

Step 2 — Second hold on same trip: { "seat_id": 2 } with same rider token

Expected Output (Step 2):

```
HTTP Status: 409 Conflict
{
  "error": {
    "code": "ALREADY_HOLDING",
    "message": "You already hold a seat on this trip.",
    "field": null
  }
}
```

Actual Output (Step 2):

```
HTTP Status: 409 Conflict
{
  "error": {
    "code": "ALREADY_HOLDING",
    "message": "You already hold a seat on this trip.",
    "field": null
  }
}
```

Result: PASS

```
tests/test_holds.py
# tests/test_holds.py
def test_rider_cannot_hold_two_seats_on_same_trip():
    """TC-07: Rider attempting a 2nd hold on the same trip gets 409 ALREADY_HOLDING."""
    coord = register("coordinator")
    rider = register("rider")
    trip = create_trip(coord["token"], total_seats=4)
    seats = seat_ids_for(trip["id"])

    client.post(
        "/holds",
        json={"seat_id": seats[0]},
        headers=auth_header(rider["token"]),
    )
    res = client.post(
        "/holds",
        json={"seat_id": seats[1]},
        headers=auth_header(rider["token"]),
    )
    assert res.status_code == 409
    assert res.json()["error"]["code"] == "ALREADY_HOLDING"
```

Section 2: User Feedback

Field	Details
Demo conducted on	August 2, 2026
Participants	4 external users (2 college students, 1 NGO volunteer coordinator, 1 patient attendant)
Demo method	Live walkthrough of the SahyogRide web app — users performed actual registration, searched trips, held seats, and confirmed bookings

User 1 — Ravi Kumar (College Student, Rider perspective)

"I registered in under a minute — no OTP verification needed which was great for a quick demo. I searched for a hospital shuttle and found the trip I was looking for. When I clicked on a seat, it immediately turned yellow showing it was 'held'. The 5-minute countdown was a smart touch — felt like booking a train ticket. Confirmed the booking in one click and got my reservation instantly. Really smooth experience overall."

Feature	Rating	Comment
Registration & Login	★★★★★	Fast, no unnecessary steps
Trip Search	★★★★☆	Works, but no date filter — had to scroll
Seat Map & Hold	★★★★★	Visual seat status was very intuitive
Confirm Booking	★★★★★	One-click, instant feedback
Overall	★★★★☆	Solid first version

Suggestions:

- Add a date filter on the search — I saw trips from last month mixed in with upcoming ones
- Show the seat number I booked on the confirmation screen

User 2 — Priya Menon (NGO Volunteer, Coordinator perspective)

"I created a shuttle trip for our weekly hospital run from our NGO's end. The form was straightforward — origin, destination, departure time, seat count, purpose. Once I submitted it, I could see the trip was live immediately. However, I had no way to see who had booked seats on my trip — that's something I really need as a coordinator. I would want to see a passenger list with names and contact info before the trip departs."

Feature	Rating	Comment
Trip Creation	★★★★★	Simple and clear form
View Trip Status	★★★★☆	Can see seats taken but not who booked
Coordinator Dashboard	★★★☆☆	No passenger list — critical missing feature
Overall	★★★★☆	Core works, but coordinator needs more control

Suggestions:

- I need to see a passenger list — names, phone numbers — before the trip
- Send a notification to all confirmed riders if I have to cancel or reschedule the trip
- Option to add a trip description or pickup point instructions

User 3 — Anita Sharma (Patient Attendant, Rider perspective)

"I was looking for a free medical shuttle for my mother's dialysis appointment. I liked that the purpose of each trip is shown — I could filter for 'medical' trips easily. The seat map was simple but I wasn't sure whether the green seat meant available or confirmed. I accidentally clicked cancel reservation while exploring — wished there was an 'Are you sure?' popup before it deleted my booking. I had to rebook again."

Feature	Rating	Comment
Trip Search by Purpose	★★★★☆	Purpose tag very helpful
Seat Map Clarity	★★★★☆	Colors need a legend/label
Cancel Reservation	★★★☆☆	No confirmation dialog — accidental cancellation happened
Overall Experience	★★★★☆	Good but needs polish for non-tech users

Suggestions:

- Add a confirmation popup before cancelling — 'Are you sure you want to cancel?'
- Show a color legend for the seat map (green = available, yellow = held, red = reserved)
- After booking, send an SMS or email with trip details

User 4 — Arjun Pillai (College Student, tried AI Search)

"I tried typing 'free hospital shuttle for tomorrow morning' in the search bar — wasn't sure it would work but it actually returned a relevant trip! That was impressive. The regular search also worked well. My only complaint is the app felt a bit slow when loading the seat map the first time. Also, once I confirmed my booking, there was no 'My Bookings' page I could go back to — I had to remember my seat number."

Feature	Rating	Comment
AI Semantic Search	★★★★★	Impressed — natural language worked
Page Load Speed	★★★★☆	Seat map was slightly slow to load
My Reservations Page	★★★☆☆	Couldn't find confirmed bookings easily
Overall	★★★★☆	AI feature was the highlight

Suggestions:

- Add a 'My Bookings' page prominently on the home screen
- Show a QR code or ticket number after booking confirmation
- Optimize seat map loading speed

Summary of All Feedback

Theme	Frequency Mentioned	Priority for Sprint 2
Need passenger list for coordinators	1/4 users	High
Date filter on trip search	2/4 users	High
Cancel confirmation dialog	2/4 users	High
My Bookings / Reservation history page	2/4 users	High
Seat map color legend	1/4 users	Medium
Email/SMS notification after booking	2/4 users	Medium

Theme	Frequency Mentioned	Priority for Sprint 2
AI search was impressive	1/4 users	Keep & improve
Page load optimization	1/4 users	Low

Section 3: Plan for Sprint 2 (Milestone 4)

Features to Implement

Priority	Feature	User Story	Notes
High	Date filter on trip search	SAHYOG-06 enhancement	Most requested by users
High	Coordinator passenger list	SAHYOG-20	Coordinator needs to see who booked
Medium	Cancel confirmation dialog (Frontend)	UX improvement	Prevent accidental cancellations
Medium	Reservation history page	SAHYOG-22	Users want to see past bookings
Low	Email notification on confirmation	Beyond scope	Nice-to-have
Low	Improve AI search accuracy	SAHYOG-26	Tune embedding threshold

Technical Improvements Planned

- Improve test coverage — add edge cases for hold expiry (SAHYOG-18 + SAHYOG-19 concurrency stress tests)
- Frontend polish — loading states on all forms, better error messages shown to users
- Validation improvements — validate departure_time is in the future on trip creation
- Documentation — update Swagger YAML with any new endpoints from Sprint 2

Sprint 2 Team Assignment

Member	Email	Task
Vibhav Shukla (Team Lead)	23f2003054@ds.study.iitm.ac.in	Sprint planning, test case review, milestone coordination
Siddharth Kumar Yadav	23f1001362@ds.study.iitm.ac.in	Passenger list endpoint + code review for all PRs
Nilesh Mehra	22f3002392@ds.study.iitm.ac.in	Date filter on search + additional backend test cases
Navneet Kumar Yadav	23f2002092@ds.study.iitm.ac.in	Reservation history page (frontend) + cancel confirmation dialog
Aakash Kumar Katha	21f1004110@ds.study.iitm.ac.in	Concurrency stress tests + AI search threshold improvements



INDIAN INSTITUTE OF TECHNOLOGY MADRAS

Software Engineering — Project Milestone Report

POST-SPRINT-1 ADDENDUM

Feature & API Test Case Update

SahyogRide

An AI-Assisted Community Ride & Seat-Booking Platform

Team	078
Team Code	MAY2026-Team-078
Team Lead	Vibhav Shukla
Extends	MAY2026-Team-078-Milestone3.pdf (Sprint 1 Report)
Addendum date	12 August 2026

This addendum documents everything built **after** the Milestone 3 / Sprint 1 report — it does not alter Sections 1–3 of that report, which remain the historical record of the Sprint 1 demo and test run. It brings the AI feature summary, API test cases, and the Sprint 2 plan up to date with the current codebase, and is generated to stay in sync with `openapi.yaml` at the repository root.

Updated: AI Features in SahyogRide

The original Milestone 3 report (page 2) listed 3 AI features. Three more have since been built, for 6 total. All remain isolated to `app/services/ai_service.py`, every call has a 5-second timeout and returns `None` on failure, and every caller falls back to a deterministic, non-AI path — the app works identically end to end with `AI_ENABLED=false`.

#	Feature	Endpoint	Description
1	Semantic Trip Search	GET/POST <code>/ai/search</code>	pgvector embeddings match a natural-language query to trips. Falls back to tokenized keyword search if AI is disabled or the call fails.
2	AI Trip Triage	Internal background task	After a reservation is confirmed, classifies rider urgency, writes a one-line AI trip summary, and flags trips with unusually high demand.
3	Embedding Backfill	<code>backfill_embeddings.py</code>	Batch script generating pgvector embeddings for existing trips.
4	AI Rider Assistant <i>(new)</i>	POST <code>/ai/assistant</code>	General "how does this app work" help — no database access at all, grounded on a fixed FAQ that also serves as the deterministic fallback.
5	AI Smart Seat Pick <i>(new)</i>	POST <code>/trips/{id}/seat-recommendation</code>	Suggests one seat from a free-text note (e.g. "wheelchair"). Read-only — the rider still holds the seat themselves. The AI's pick is re-validated against currently available seats before being trusted; a hallucinated pick falls back identically to AI-off.
6	AI Accessibility Note Classification <i>(new)</i>	Internal background task	Classifies a rider's free-text accessibility note (captured at booking) into one tag for the coordinator's passenger list.

Not a live AI call: GET `/trips/{id}/similar` (similar-trip suggestions) ranks trips by cosine distance against the already-populated `trips.embedding` column — pure pgvector math, zero additional AI network calls. It is deliberately not counted in the "6 AI features" above for that reason.

All 6 features, plus similar-trip suggestions, are fully documented — request/response schemas, error codes, and examples — in `openapi.yaml` at the repository root.

Section 4: Additional API Test Cases (Post-Sprint-1 Features)

Note: TC-08 to TC-13 cover features built after the Sprint 1 test run (TC-01-TC-07, Section 1). All 6 pass. Continuing the same table format as Section 1 — Field/Details, Input, Expected Output, Actual Output, Result, and the real pytest source.

TC-08 — Coordinator Sets Bus Stops & Live Vehicle Position

API Endpoint	PUT /trips/{id}/stops, PATCH /trips/{id}/stops/current
User Story	SAHYOG-46: Coordinator bus stops & rider-facing live position

Input — set the route:

```
{
  "stop_names": [
    "City Hospital (boarding)",
    "Market Square",
    "Town Hall",
    "Railway Station (destination)"
  ]
}
```

Actual Output — set the route (HTTP 200):

```
[
  { "id": 5, "name": "City Hospital (boarding)", "sequence": 0 },
  { "id": 6, "name": "Market Square", "sequence": 1 },
  { "id": 7, "name": "Town Hall", "sequence": 2 },
  { "id": 8, "name": "Railway Station (destination)", "sequence": 3 }
]
```

Input — mark the vehicle's current stop:

```
{ "sequence": 1 }
```

Actual Output (HTTP 200):

```
{
  "stops": [ ...same 4 stops as above... ],
  "current_stop_sequence": 1
}
```

Result: PASS

tests/test_bus_stops.py

```
def test_set_stops_success():
    # TC-08: Coordinator replaces a trip's entire route in one call.
    coordinator = register("coordinator")
    trip = create_trip(coordinator["token"])

    res = client.put(
        f"/trips/{trip['id']}/stops",
        json={"stop_names": ROUTE},
        headers=auth_header(coordinator["token"]),
    )
    assert res.status_code == 200
    stops = res.json()
    assert [s["name"] for s in stops] == ROUTE
    assert [s["sequence"] for s in stops] == [0, 1, 2, 3]

def test_set_stops_forbidden_for_rider():
    # A rider may never configure a coordinator's route.
    coordinator = register("coordinator")
    rider = register("rider")
    trip = create_trip(coordinator["token"])

    res = client.put(
        f"/trips/{trip['id']}/stops",
        json={"stop_names": ROUTE},
        headers=auth_header(rider["token"]),
    )
    assert res.status_code == 403
```

TC-09 — Coordinator Passenger List with AI Urgency, Accessibility Tag & Digest

API Endpoint	GET /trips/{id}/passengers
User Story	SAHYOG-20, SAHYOG-30, SAHYOG-32, SAHYOG-40, SAHYOG-43

Actual Output — one confirmed passenger, name/phone/note captured at booking (HTTP 200):

```
{
  "passengers": [
    {
      "reservation_id": 2,
      "rider_name": "Test User",
      "seat_number": "1",
      "confirmed_at": "2026-08-12T15:14:30.797713Z",
      "ai_urgency_label": null,
      "ai_accessibility_tags": null,
      "passenger_name": "Asha Rao",
      "passenger_phone": "9998887777"
    }
  ],
  "digest": null
}
```

Note: `ai_urgency_label`, `ai_accessibility_tags`, and `digest` are null above because this environment has no `AI_API_KEY` configured — per CLAUDE.md rule 3, that must never break the endpoint, and it doesn't (still HTTP 200). The test below proves the classification path itself with a mocked AI response, since it can't be exercised live without a real key.

Result: PASS

tests/test_accessibility_notes.py

```
def test_note_classified_into_tag_when_ai_succeeds(monkeypatch):
    # TC-09b: proves the classification path with AI mocked "on".
    fake_result = AccessibilityTagResult(tag="wheelchair")
    monkeypatch.setattr(ai_service, "classify_accessibility_note", lambda note: fake_result)

    coordinator = register("coordinator")
    rider = register("rider")
    trip = create_trip(coordinator["token"])
    seat_id = seat_ids_for(trip["id"])[0]

    reservation = hold_and_confirm(rider["token"], seat_id, notes="I use a wheelchair")

    db = SessionLocal()
    try:
        row = db.get(Reservation, reservation["id"])
        assert row.ai_accessibility_tags == "wheelchair"
    finally:
        db.close()
```

TC-10 — AI Seat Recommendation Never Trusts a Hallucinated/Unavailable Seat

API Endpoint	POST /trips/{id}/seat-recommendation
User Story	SAHYOG-38: AI smart seat pick

Input:

```
{ "note": "I use a wheelchair" }
```

Actual Output — AI unavailable in this environment, deterministic fallback (HTTP 200):

```
{
  "seat_number": "1",
  "reason": "Nearest available seat (AI suggestion unavailable).",
  "fallback": true
}
```

Result: PASS

tests/test_seat_recommendation.py

```
def test_recommendation_falls_back_when_ai_picks_an_unavailable_seat(monkeypatch):
    # TC-10: hallucinated / stale seat_number that isn't in the trip's
    # actual available set - must never be trusted or returned as-is.
    fake_result = SeatRecommendationResult(seat_number="999", reason="Made up.")
    monkeypatch.setattr(ai_service, "recommend_seat", lambda note, seats: fake_result)

    coordinator = register("coordinator")
    rider = register("rider")
    trip = create_trip(coordinator["token"], total_seats=3)

    res = client.post(
        f"/trips/{trip['id']}/seat-recommendation",
        json={"note": "anything"},
        headers=auth_header(rider["token"]),
    )
    assert res.status_code == 200
    body = res.json()
    assert body["fallback"] is True
    assert body["seat_number"] == "1" # lowest-numbered available seat
```

TC-11 — AI Rider Assistant Answers Without Any Database Access

API Endpoint	POST /ai/assistant
User Story	SAHYOG-45: AI help assistant

Input:

```
{ "question": "how can I track the vehicle?" }
```

Actual Output (HTTP 200, deterministic FAQ fallback in this environment):

```
{
  "answer": "Once your seat is confirmed, check the 'Live vehicle tracker' card on the
confirmation page or on 'My reservations' - it shows the route and updates
automatically every few seconds as the coordinator marks which stop the
vehicle is currently at...",
  "fallback": true
}
```

Result: PASS

tests/test_rider_assistant.py

```
def test_assistant_answers_vehicle_tracking_question_via_fallback():
    rider = register("rider")
    res = client.post(
        "/ai/assistant", json={"question": "how can I track the vehicle?"},
        headers=auth_header(rider["token"])
    )
    assert res.status_code == 200
    body = res.json()
    assert body["fallback"] is True
    assert "tracker" in body["answer"].lower() or "track" in body["answer"].lower()
```

TC-12 — Similar Trips Falls Back to Same-Destination When No Embedding

API Endpoint	GET /trips/{id}/similar
User Story	SAHYOG-39: AI similar-trip suggestions

Actual Output — target trip has no embedding yet (HTTP 200):

```
{
  "trips": [
    { "id": 4, "origin": "General Hospital", "destination": "Railway Station",
      "departure_time": "2026-08-16T09:30:00Z", "total_seats": 4,
      "seats_available": 4, "purpose": "medical" }
  ],
  "fallback": true
}
```

Result: PASS

tests/test_similar_trips.py

```
def test_similar_trips_ranks_by_embedding_distance():
    # TC-12b: with embeddings set, ranks by cosine distance and never
    # suggests the trip itself.
    coordinator = register("coordinator")
    rider = register("rider")
    target = create_trip(coordinator["token"], destination=f"Dest-{uuid.uuid4().hex[:8]}")
    close = create_trip(coordinator["token"], destination=f"Dest-{uuid.uuid4().hex[:8]}")
    far = create_trip(coordinator["token"], destination=f"Dest-{uuid.uuid4().hex[:8]}")

    idx = random_index()
    set_embedding(target["id"], unit_vector(idx))
    set_embedding(close["id"], unit_vector(idx)) # identical -> distance 0
    set_embedding(far["id"], unit_vector((idx + 1) % VEC_DIM)) # orthogonal -> outside threshold

    res = client.get(f"/trips/{target['id']}/similar", headers=auth_header(rider["token"]))
    assert res.status_code == 200
    body = res.json()
    assert body["fallback"] is False
    ids = [t["id"] for t in body["trips"]]
    assert close["id"] in ids
    assert far["id"] not in ids
    assert target["id"] not in ids # never suggests itself
```

TC-13 — Confirm Reservation Requires a Reachable Passenger

API Endpoint	POST /reservations
User Story	SAHYOG-43: Capture passenger contact details at booking time

Bug context:

Before SAHYOG-43, a reservation only needed a valid `hold_id` — a coordinator had a confirmed seat but no way to reach the actual traveller if the trip changed. `passenger_name` and `passenger_phone` are now required fields.

Input (missing passenger_phone):

```
{ "hold_id": 3, "passenger_name": "Asha Rao" }
```

Actual Output (HTTP 422):

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Field required",
    "field": "passenger_phone"
  }
}
```

Result: PASS

tests/test_reservations.py (pattern)

```
def test_confirm_reservation_requires_passenger_phone():
    coordinator = register("coordinator")
    rider = register("rider")
    trip = create_trip(coordinator["token"])
    seat_id = seat_ids_for(trip["id"])[0]
    hold = client.post("/holds", json={"seat_id": seat_id}, headers=auth_header(rider["token"]))

    res = client.post(
        "/reservations",
        json={"hold_id": hold.json()["id"], "passenger_name": "Asha Rao"},
        headers=auth_header(rider["token"]),
    )
    assert res.status_code == 422
    assert res.json()["error"]["field"] == "passenger_phone"
```

Section 3 Revisited: Sprint 2 Plan — Status Update

The original Section 3 (page 14) listed the Sprint 2 plan. Status against the current codebase:

Priority	Feature	Status	Notes
High	Date filter on trip search	Done	GET /trips?date=
High	Coordinator passenger list	Done	GET /trips/{id}/passengers — plus urgency/accessibility badges and an AI digest, beyond the original scope
Medium	Cancel confirmation dialog (frontend)	Not yet	Cancel still fires immediately on click — no "are you sure?" step
Medium	Reservation history page	Done	"My reservations" — includes live vehicle tracker per booking
Low	Email notification on confirmation	— Out of scope	CLAUDE.md explicitly excludes SMS/email
Low	Improve AI search accuracy	Done	Keyword fallback now tokenizes natural-language queries instead of requiring a whole-phrase match

Built beyond the original Sprint 2 scope (see the updated AI features table):

- AI smart seat pick, AI accessibility note classification, AI rider assistant, similar-trip suggestions
- Coordinator-managed bus stops and live vehicle position, with a rider-facing tracker and zoomed map view
- Optional origin/destination map on trip pages (client-side geocoding)
- Passenger name/phone capture at booking time

See `openapi.yaml` at the repository root for the complete, current API contract — every endpoint, request/response schema, and error code referenced above.